

2026 edition

Deep Learning for Music Analysis and Generation

# Introduction

Intros + course logistics



**Yi-Hsuan Yang** Ph.D.

yhyangtw@ntu.edu.tw

# Course Website

<https://cool.ntu.edu.tw/courses/65378>

<https://affige.github.io/teaching.html>

<https://github.com/affige/DeepMIR/>

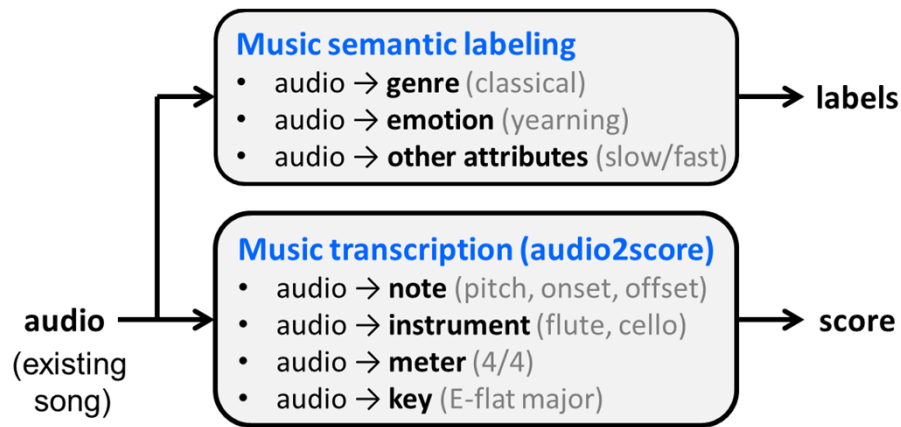


# Outline

- **Intros of Music & AI**
- Course logistics
- Fundamentals of musical audio
- Appendix: ML 101

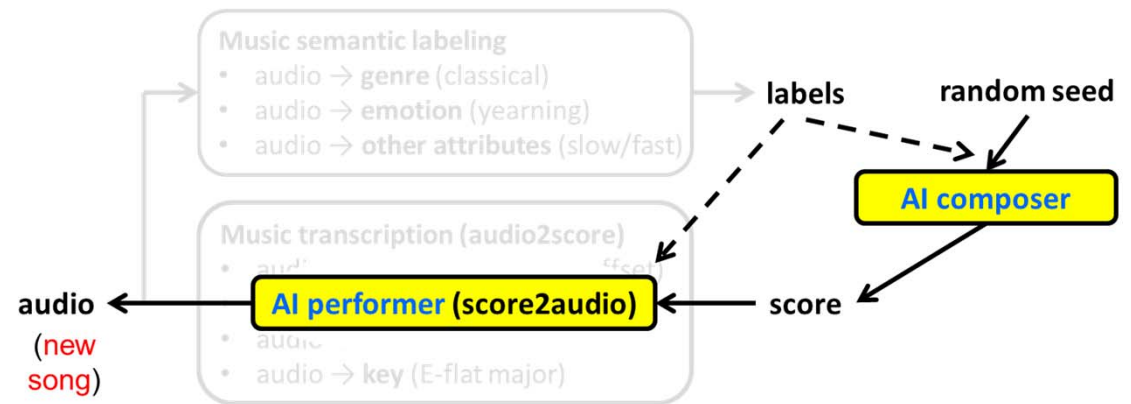
# Music AI; or *Music Information Research* (MIR)

- Music analysis



- music understanding
- music search
- music recommendation

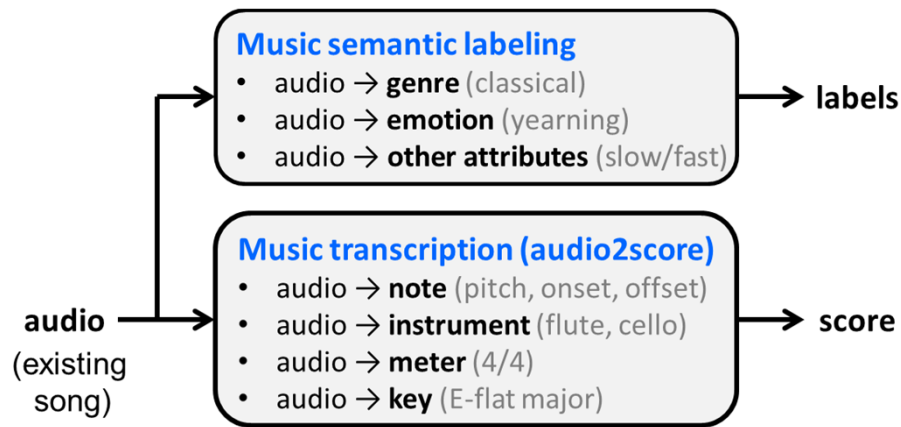
- Music generation



- MIDI generation
- audio generation
- MIDI-to-audio generation

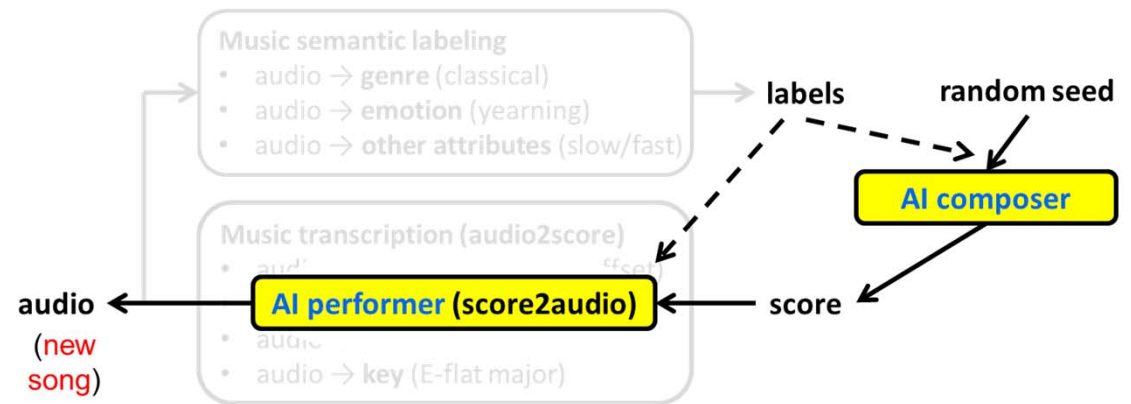
# Music AI: {signal processing, machine learning} + music

- Music analysis



- music understanding
- music search
- music recommendation

- Music generation



- MIDI generation
- audio generation
- MIDI-to-audio generation

# Music Analysis Demo 1: Music Transcription

<https://songscription.ai/>



- songscription.ai
- AnthemScore
- Melody Scanner
- Klangio

**Flight of the Bumblebee**  
Nikolai Rimsky-Korsakov

Up Down Scale  
Exercise

Invention No. 4 in D Minor  
Johann Sebastian Bach

**Flight of the Bumblebee**  
Nikolai Rimsky-Korsakov

Original Audio

0:00 / 0:22

Play Generated Sheet Music

Flight-of-the-bumblebee-piano

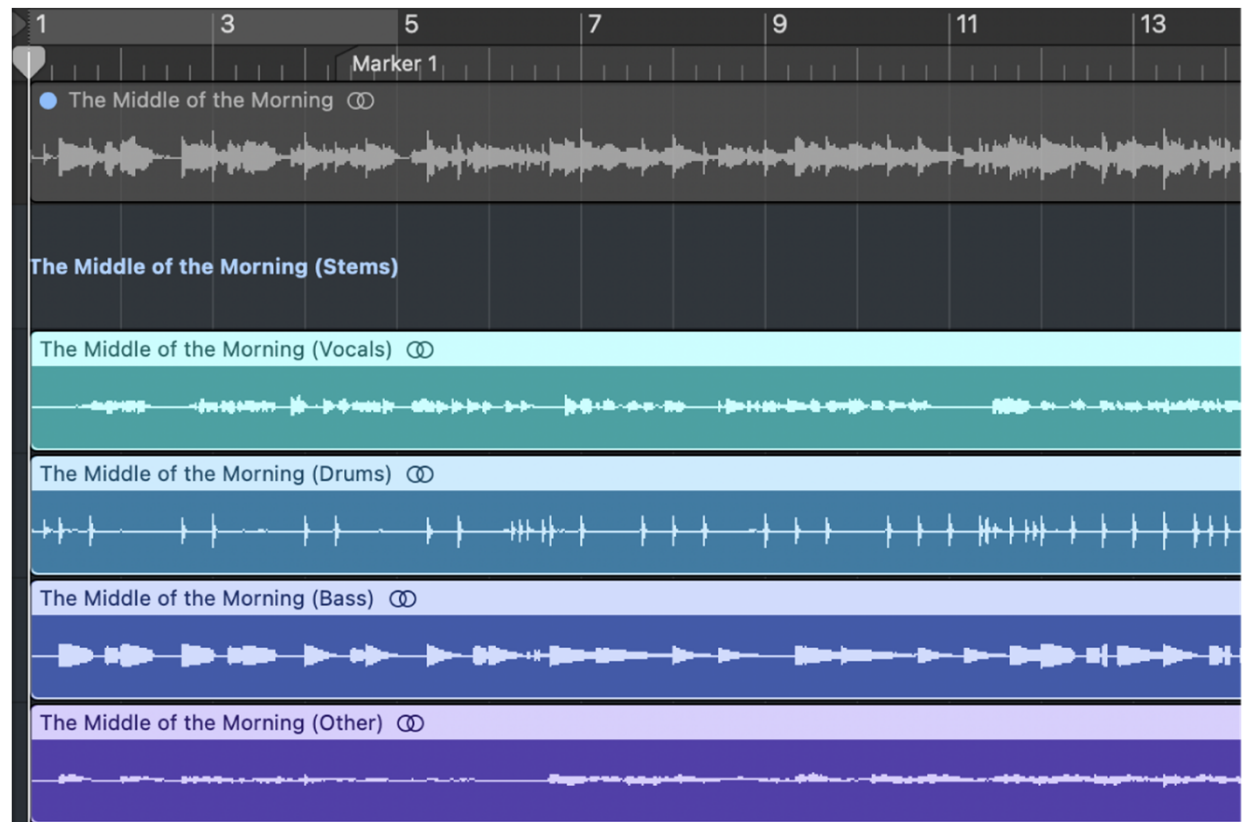
The image displays a musical score for the piano version of 'Flight of the Bumblebee' by Nikolai Rimsky-Korsakov. The score is written for piano and features a treble and bass staff. The melody is characterized by rapid, ascending and descending runs, typical of the piece. The score is presented in a clean, modern layout with a green header and a white background for the music.

# Music Analysis Demo 2: Source Separation

<https://www.youtube.com/watch?v=r4NcZbgtlgw>



- Apple Logic Pro
- iZotope RX
- Gaudio Studio
- RipX
- moises.ai
- lalal.ai



Source: <https://support.apple.com/en-md/guide/logicpro/lgcp61bae908/mac>

# Music Analysis Demo 3: Audio Understanding

<https://research.nvidia.com/labs/adlr/AF3/>



Audio Flamingo 3

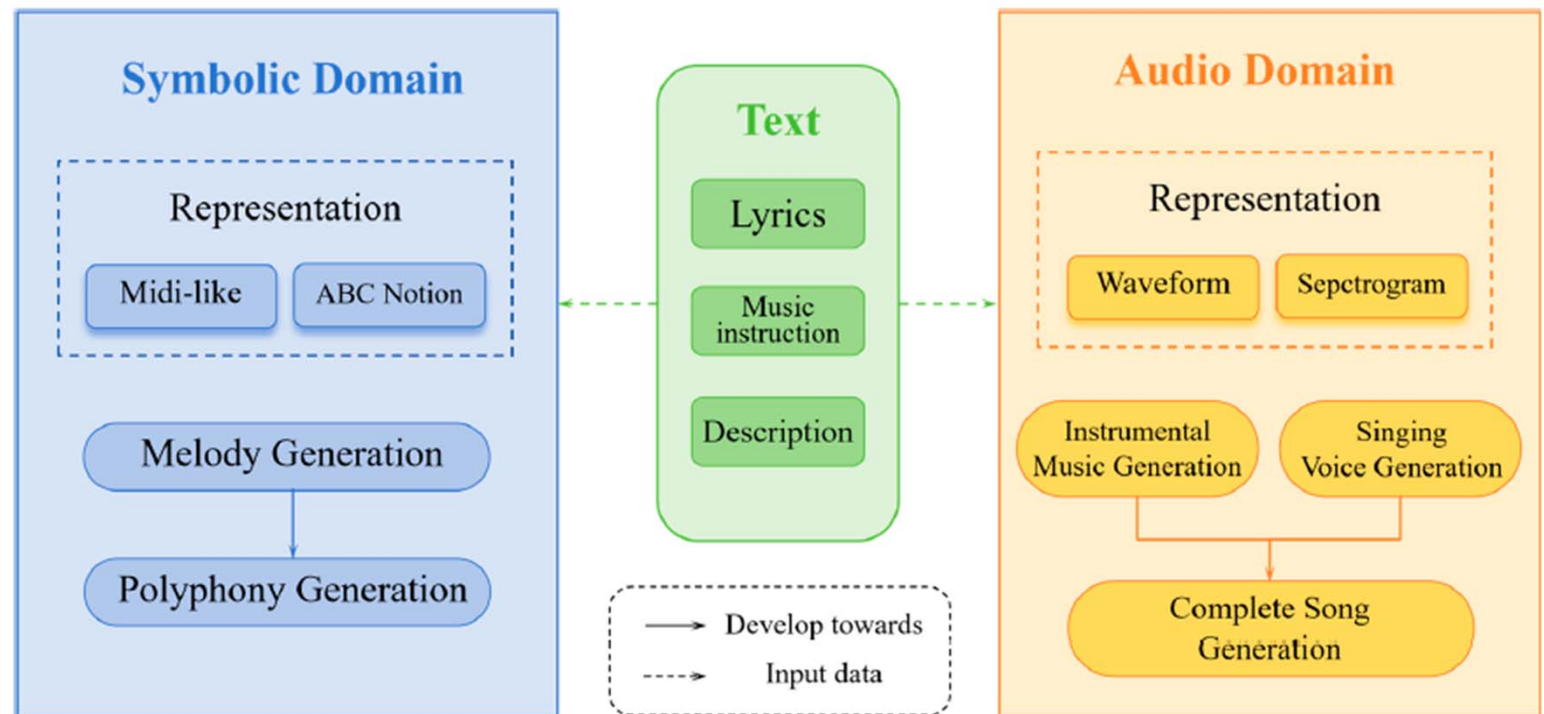
| Audio                                                                               | Instruction                                                                             | Ground-truth                                                      | Audio Flamingo 3                                                                                                |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
|    | What is surprising about the relationship between the barking and the music?            | Dogs barking in sync with the music                               | The <b>barking and music are synchronized</b> , creating a rhythmic and harmonious interaction.                 |
|    | What makes this saxophone performance unlike typical saxophone sounds?                  | Aside from "Interstellar Spaces", Saxophones do not bark nor meow | The <b>saxophone is played in a way that resembles a dog howling</b> , creating a unique and unusual sound.     |
|  | What makes this combination of acoustic and ambient sounds unlikely to appear together? | Unlikely existence of banjo and rain sounds at the same time.     | The <b>contrast between the lively music and the sound of raindrops</b> creates an unusual auditory experience. |

# Positive Use Cases of Music Generation

- **Make music easier to play with** (i.e., for common people)
  - “democratization” of music creation
- **Make musicians’ life easier** (i.e., for musicians)
  - inspire ideas
  - suggest continuations
  - suggest accompaniments
- **Create copyright free music for videos or games**
- **Music education / learning**
- **Digital archival**

# Evolution of Music Generation Research

- Monophonic MIDI (melody) → polyphonic MIDI (e.g. piano)
- Single-instrument MIDI → multi-instrument MIDI
- MIDI → audio
- Solo → full song

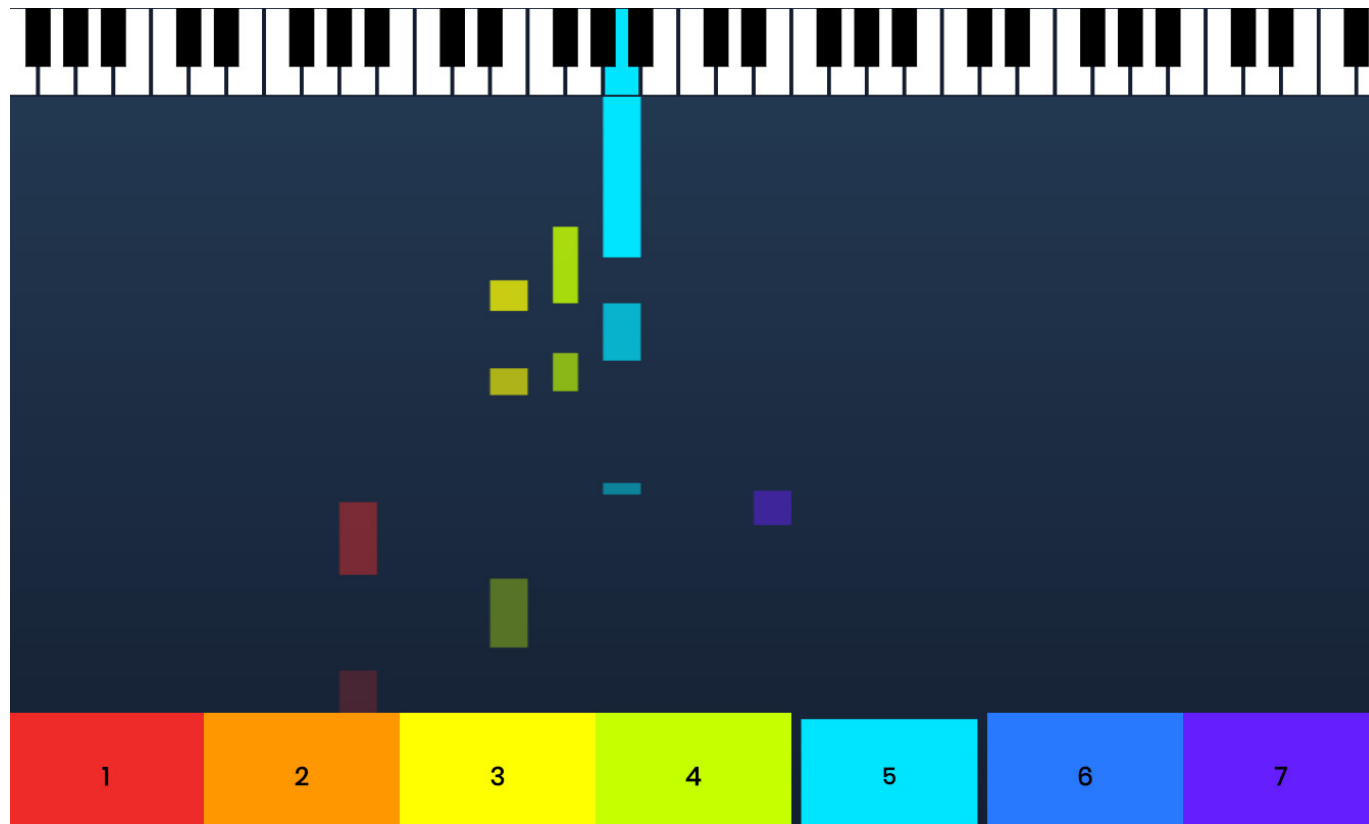


Source: <https://www.mdpi.com/2079-9292/14/6/1197>

# Music Generation Demo 1: Piano Genie

(Make music easier to play with)

<https://magenta.tensorflow.org/pianogenie>



# Music Generation Demo 2: Tone Transfer

(Make music easier to play with)

<https://sites.research.google/tonetransfer>



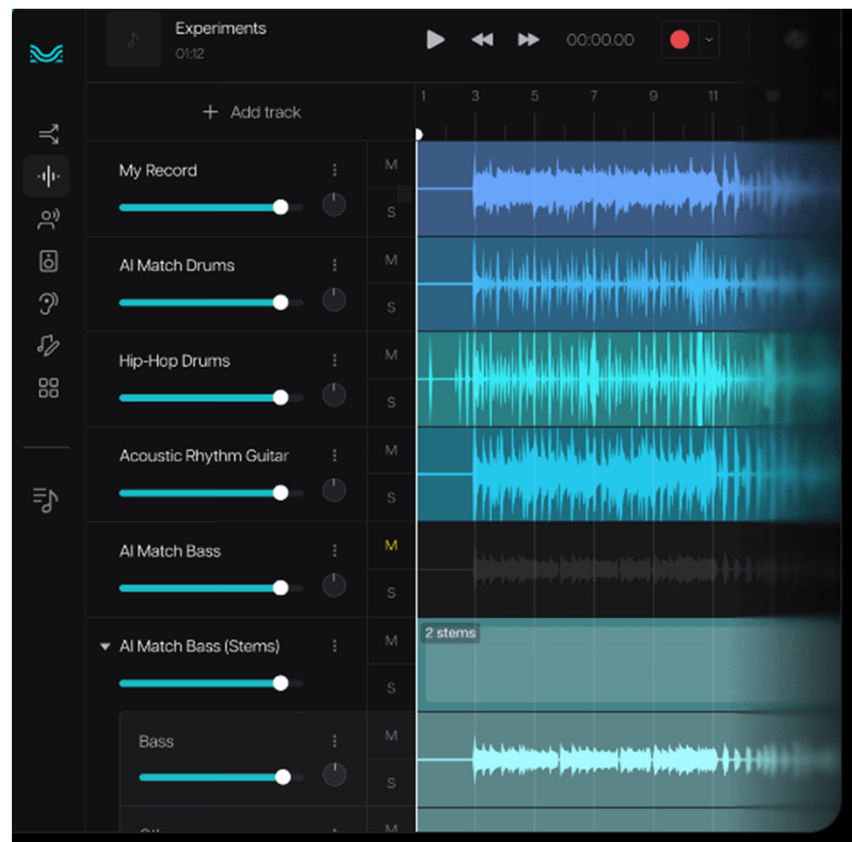
# Music Generation Demo 3: Song Generation

(Create copyright free music; for musicians)

<https://www.youtube.com/watch?v=MCLVhRjlvys>



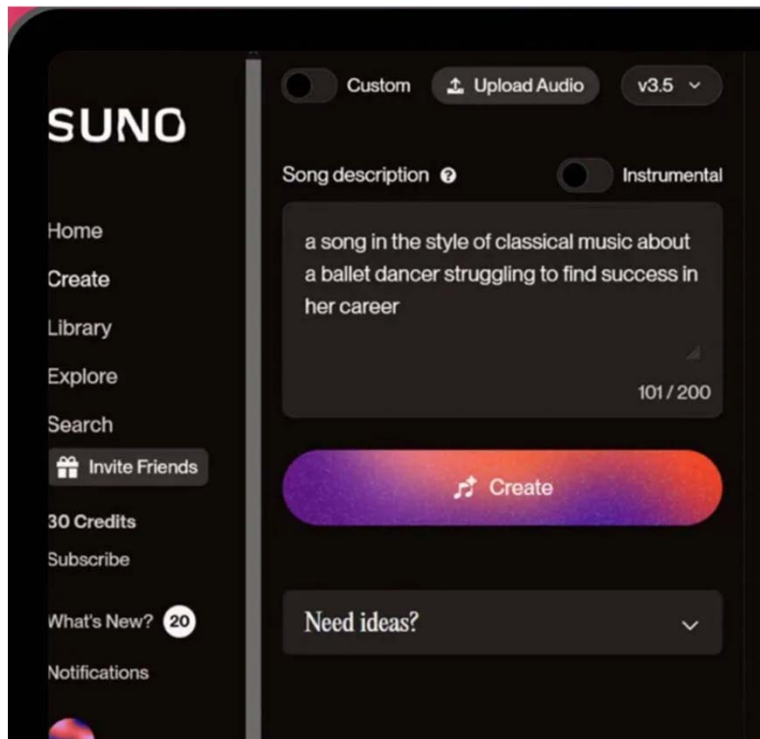
**moises**



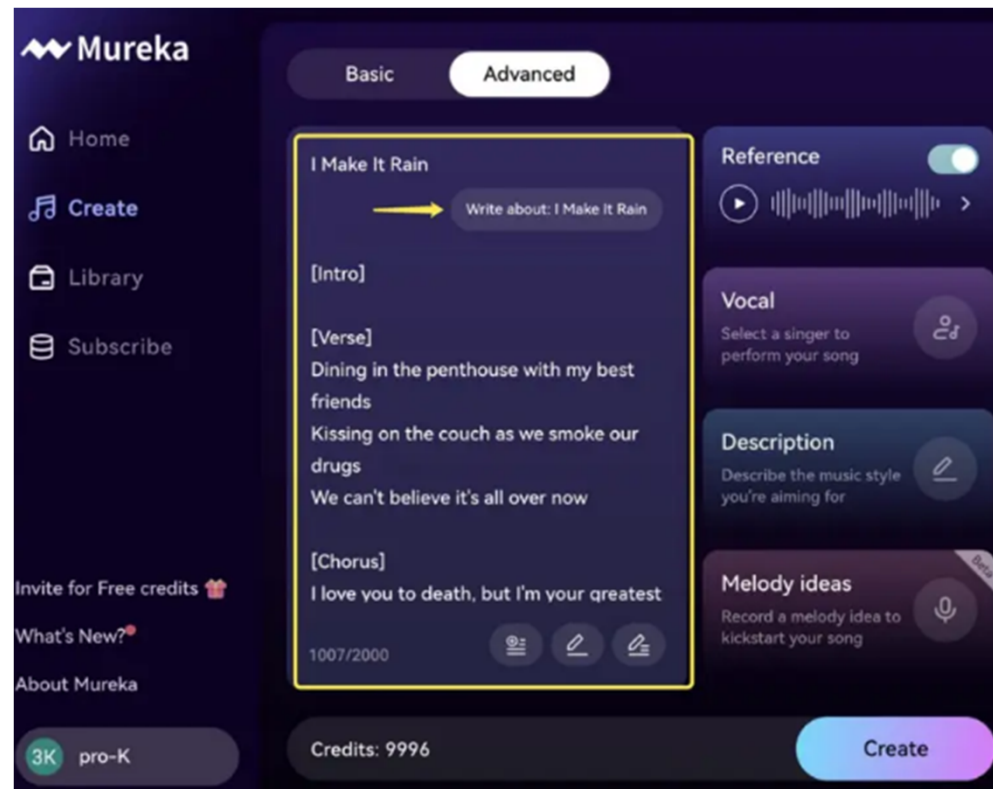
# Music Generation Demo 4: Song Generation

(Create copyright free music; for common people)

<https://suno.com/song/722aa074-5010-44d1-ad51-55468f22abc2>



<https://suno.com/>



<https://www.mureka.ai/>

# Global Interest in Music AI: Industry



(Slide from Rujing Huang, Bob L. T. Sturm, and Andre Holzapfel, “De-centering the West: East Asian Philosophies and the Ethics of Applying Artificial Intelligence to Music,” ISMIR 2021)

## Global Interest in Music AI: Academia

- MIT EECS | Anna Huang (<https://czhuang.github.io/>)
- CMU CS | Chris Donahue (<https://chrisdonahue.com/>)
- U Michigan SMTD | Hao-Wen Dong (<https://hermandong.com/>)
- Stanford CCRMA | Ge Wang (<https://ccrma.stanford.edu/~ge/>)
- NYU MARL | Brian McFee (<https://brianmcfree.net/>)
- U Rochester | Zhiyao Duan (<https://hajim.rochester.edu/ece/sites/zduan/>)
- Georgia Tech | Alexander Lerch (<https://www.alexanderlerch.com/>)
- Centre for Digital Music, Queen Mary University of London  
(<https://www.c4dm.eecs.qmul.ac.uk/>)
- Academia Sinica | Li Su (<https://homepage.iis.sinica.edu.tw/pages/lisu/>)

# Outline

- Intros of Music & AI
- **Course logistics**
- Fundamentals of musical audio
- Appendix: ML 101

# This Course: Prerequisites

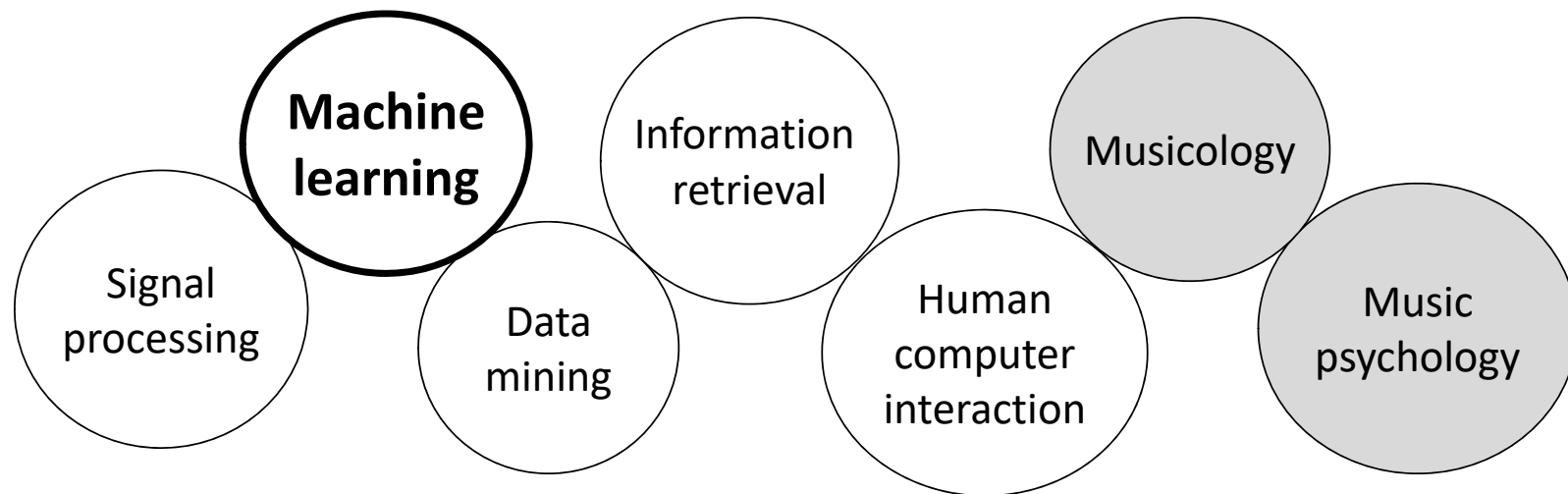
- **Graduate level (CommE5070) @ NTU GICE**
  - It's **NOT** a music course
  - It's an EE/CS graduate-level course working on music data/problems
- Suitable for people who have
  - Great interest in music
  - Good background in machine learning & deep learning
  - Good coding experience in python and a deep learning framework such as PyTorch
- Not a class designed for deep learning beginners

# This Course: Wills and Won'ts

- Will talk about
  - Domain knowledge in music data representation
  - Domain knowledge in music analysis: timbre, rhythm, pitch
  - Deep learning-based music analysis
  - Deep learning-based audio generation
  - Deep learning-based MIDI sequence generation
- **Won't** talk about (too much on)
  - Basics in machine learning and deep learning
  - Applications in other domains

## This Course: Objective

- Get you ready to do **ML** research on music AI
- Lots of hands-on



# Lecturer

- Lecturer
  - Yi-Hsuan Yang (楊奕軒)
    - <https://affige.github.io/>
    - [yhyangtw@ntu.edu.tw](mailto:yhyangtw@ntu.edu.tw)
- Office hour
  - Tuesday 9:30-11:30, or by appointment
  - Office: BL-616 (電二)

# Teaching Assistants

- TAs
  - Ting-Yi Hu (胡庭翊)
    - [tingyihu521@gmail.com](mailto:tingyihu521@gmail.com)
  - Bo-Rui Chen (陳柏睿)
    - [rakechen168@gmail.com](mailto:rakechen168@gmail.com)
- Office hour
  - Monday 13:30-15:00, or by appointment
  - Office: BL-505 (博理館)

## Location & Time

- Location: 電二229
- Time: Tuesday 6,7,8
- Usually
  - 6\*: 13:40-14:30 (i.e., 20 mins later)
  - 7\*: 14:40-15:30
  - 8\*: 15:40-16:30
- Last week (W16)
  - 6: 13:20- (we will begin the class on time for the last week)

## Online Option

- Live through Google Meet: <http://meet.google.com/dka-enxo-toy>

# Textbook

(for the music analysis part)

- Reference textbook

Meinard Müller

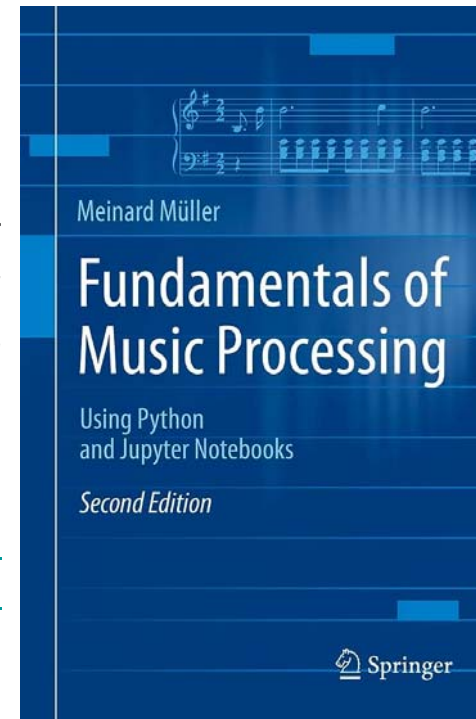
*Fundamentals of Music Processing*  
Using Python and Jupyter Notebooks

ISBN: 978-3-030-69808-9

Springer, April 2021

<https://www.audiolabs-erlangen.de/fau/professor/mueller/bookFMP>

<https://www.audiolabs-erlangen.de/resources/MIR/FMP/C0/C0.html>



- Related book

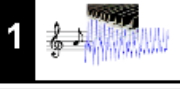
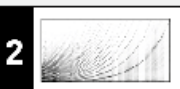
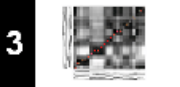
- *An Introduction to Audio Content Analysis: Applications in Signal Processing and Music Informatics*, Wiley

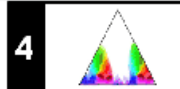
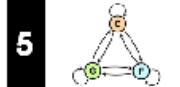
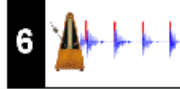
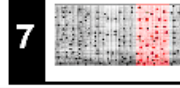
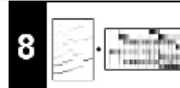
<https://github.com/alexanderlerch/pyACA>

<https://github.com/alexanderlerch/ACA-Slides>

# FMP Notebook

<https://www.audiolabs-erlangen.de/resources/MIR/FMP/C0/C0.html>

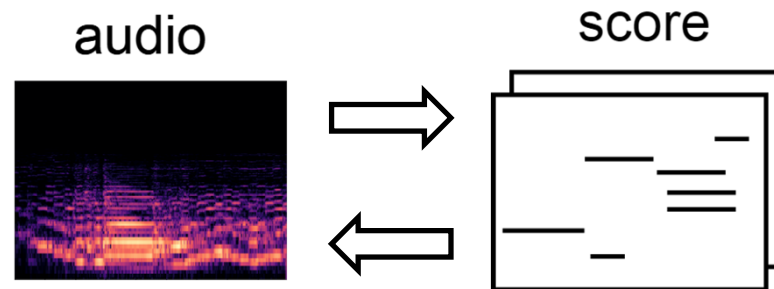
| Part                                                                               | Title                                       | Notions, Techniques & Algorithms                                                                                                          | HTML                   | IPYNB                   |
|------------------------------------------------------------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------------------|
|    | <a href="#">Basics</a>                      | Basic information on Python, Jupyter notebooks, Anaconda package management system, Python environments, visualizations, and other topics | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|    | <a href="#">Overview</a>                    | Overview of the notebooks ( <a href="https://www.audiolabs-erlangen.de/FMP">https://www.audiolabs-erlangen.de/FMP</a> )                   | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|    | <a href="#">Music Representations</a>       | Music notation, MIDI, audio signal, waveform, pitch, loudness, timbre                                                                     | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|  | <a href="#">Fourier Analysis of Signals</a> | Discrete/analog signal, sinusoid, exponential, Fourier transform, Fourier representation, DFT, FFT, STFT                                  | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|  | <a href="#">Music Synchronization</a>       | Chroma feature, dynamic programming, dynamic time warping (DTW), alignment, user interface                                                | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |

| Part                                                                                  | Title                                                  | Notions, Techniques & Algorithms                                                                                                                               | HTML                   | IPYNB                   |
|---------------------------------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------------------------|
|    | <a href="#">Music Structure Analysis</a>               | Similarity matrix, repetition, thumbnail, homogeneity, novelty, evaluation, precision, recall, F-measure, visualization, scape plot                            | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|    | <a href="#">Chord Recognition</a>                      | Harmony, music theory, chords, scales, templates, hidden Markov model (HMM), evaluation                                                                        | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|    | <a href="#">Tempo and Beat Tracking</a>                | Onset, novelty, tempo, tempogram, beat, periodicity, Fourier analysis, autocorrelation                                                                         | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|   | <a href="#">Content-Based Audio Retrieval</a>          | Identification, fingerprint, indexing, inverted list, matching, version, cover song                                                                            | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |
|  | <a href="#">Musically Informed Audio Decomposition</a> | Harmonic/percussive separation, signal reconstruction, instantaneous frequency, fundamental frequency (F0), trajectory, nonnegative matrix factorization (NMF) | <a href="#">[html]</a> | <a href="#">[ipynb]</a> |

# Grading Policy

- Grading policy
  - **Assignments** (60%), 3 times, each 20%
  - **Final Project** (40%): for teams of 2 or **3** (recommended)
- Work reasonably hard to get a high score

# Course Material



## Topics

- **audio** → **audio**: signal processing
- **audio** → **score**: transcription
- **score** → **score**: composition
- **score** → **audio**: synthesis
- **audio** → **knowledge**: audio analysis
- **score** → **knowledge**: symbolic-domain analysis

# Syllabus

- **W1.** Intros & fundamentals of musical audio
- **W2.** Music classification & transcription
  - **HW1:** Music classification
- **W3.** Music foundation models & audio language models
- **W4.** Source separation & vocoders
- **W5.** Symbolic music generation I
  - **HW2:** MIDI generation
- **W6.** Symbolic music generation II
- **W7.** Musical Audio Generation I
- **W8.** Musical Audio Generation II
  - **HW3:** Text-to-music generation
- **W9.** Musical Audio Generation III
- **W10.** Musical Audio Generation IV
- **W11.** Effect modeling & automatic mixing
- **W12. Project pitch** (proposals of final projects)
- **W13.** Miscellaneous Topics
- **W14.** Miscellaneous Topics
- (W15. Break)
- **W16. Oral presentation** of final projects

# Assignments

- **Programming (in python) + report (in English)**
  - We assume that you are comfortable with programming in python and deep learning frameworks such as PyTorch
  - One new assignment **every three weeks** (W2, W5, W8); each due in 3 weeks (W5, W8, W11)
  - Submit **code + model + report**
  - Will select around 3 people to share their work
  - NO cheating: Will run *plagiarism detector*
- **Topics**
  - HW1: music classification
  - HW2: MIDI generation
  - HW3: text-to-music audio generation

# Assignments

|     |                                                           | HW1            | HW2             | HW3           | Final project |
|-----|-----------------------------------------------------------|----------------|-----------------|---------------|---------------|
|     |                                                           | Classification | MIDI generation | Text-to-music |               |
| W1  | Intros + course logistics & fundamentals of musical audio |                |                 |               |               |
| W2  | Music classification & transcription                      | announced      |                 |               |               |
| W3  | Music foundation models & audio language models           | due            |                 |               |               |
| W4  | Source Separation & vocoders                              |                |                 |               |               |
| W5  | Symbolic music generation I                               |                | announced       |               |               |
| W6  | Symbolic music generation II                              |                | due             |               |               |
| W7  | Musical Audio Generation I: codec models                  |                |                 |               |               |
| W8  | Musical Audio Generation II: latent models                |                | due             | announced     |               |
| W9  | Musical Audio Generation III: lyrics-to-song generation   |                |                 | due           |               |
| W10 | Musical Audio Generation IV: cover-gen & MIDI-to-audio    |                |                 |               | team up       |
| W11 | Effect modeling & automatic mixing                        |                |                 |               |               |
| W12 | Project pitch                                             |                |                 |               | proposal      |
| W13 | Miscellaneous topics                                      |                |                 |               |               |
| W14 | Miscellaneous topics                                      |                |                 |               |               |
| W15 | break                                                     |                |                 |               |               |
| W16 | Final presentation                                        |                |                 |               | oral report   |
| W17 |                                                           |                |                 |               |               |
| W18 |                                                           |                |                 |               | paper report  |

# NO Cheating

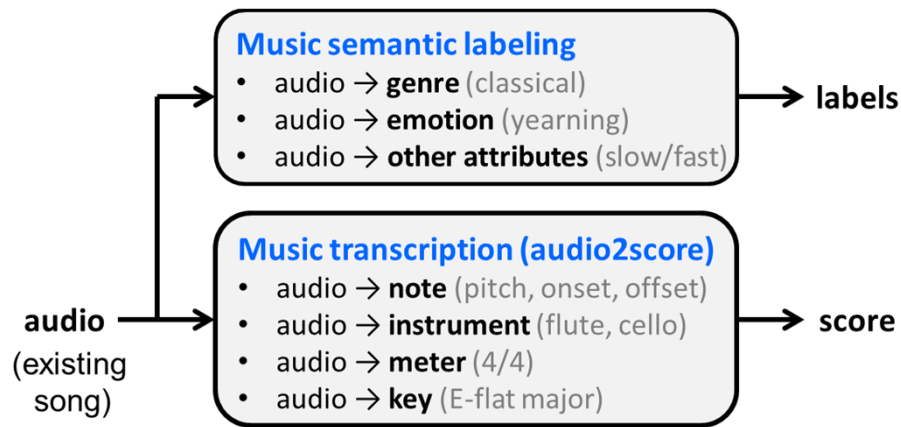
- Once caught: **failure** of the course

# Final Project

- For teams of 2 or **3** (recommended)
- Start earlier & form teams
- Deadline for **team-up**: W10
- **Project pitch**: W12
- **Final presentation**: W16
- Deadline for **final report**: W16+2

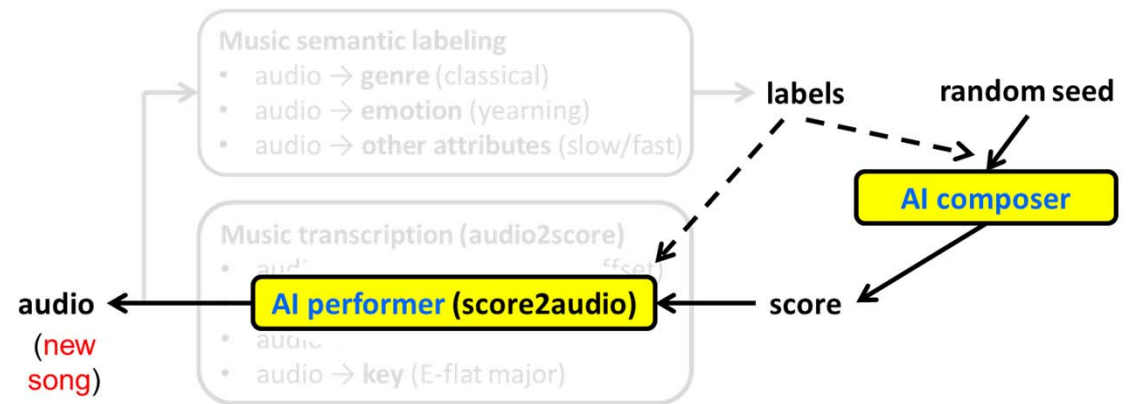
# Final Project

- Music analysis



- music semantic labeling
- music transcription
- source separation

- Music generation



- MIDI generation
- audio generation
- MIDI-to-audio generation

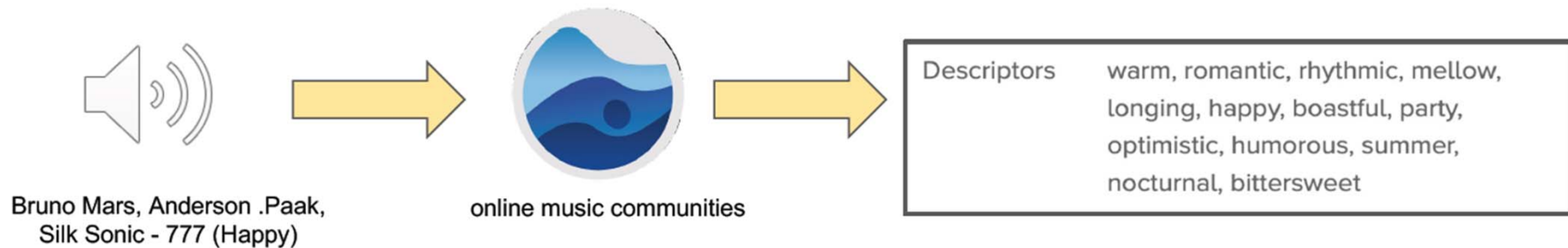
# Final Project Showcase

[https://affige.github.io/teaching\\_deepmir25.html](https://affige.github.io/teaching_deepmir25.html)

- **List of final projects** presented by the class students
  - *BLT: An IPA-Aware Agentic Framework for Controllable Lyric Translation*
    - extensions of this project has been accepted for publication at [EMNLP'26](#): "IPA-Verified Singable Lyrics Translation with LLM Agents"
  - *High-level Semantic Direction Discovery*
    - extensions of this project has been accepted for publication at [KDD'26](#): "AnchorSteer: Self-discovered concept injection for structure-preserving music editing"
  - *Style Transfer of Audio Effects with Differentiable Signal Processing*
  - *Unified Audio and Text-Controlled Automatic Music Mixing via CLAP Embeddings*
    - extensions of this project still ongoing
  - *Make Spoken Language Models Sing: A Preliminary Study using Text-Aligned Tokenization*
    - extensions of this project still ongoing

# Final Project Showcase

- Multimodal music emotion recognition: audio + lyrics + MIDI



| Song                                                 | Model            | Prediction |
|------------------------------------------------------|------------------|------------|
| Bruno Mars, Anderson .Paak, Silk Sonic - 777 (Happy) | CRNN             | Happy      |
|                                                      | MERT             | Happy      |
|                                                      | Whisper + BERT   | Angry      |
|                                                      | Late fusion      | Angry      |
|                                                      | LSTM-Attn + Remi | Angry      |

# Final Project Showcase

- Guitar effect removal

[https://y10ab1.github.io/guitar\\_effect\\_removal/](https://y10ab1.github.io/guitar_effect_removal/)



## Final Project Showcase

- Transfer music to failed recorder style  
<https://navi0105.github.io/demo/music2fail/>

# genmusic\_demo\_list

[https://github.com/affige/genmusic\\_demo\\_list](https://github.com/affige/genmusic_demo_list)

## About

a list of demo websites for automatic  
music generation research

artificial-intelligence

music-generation

# Resources

- ML/DL
  - <http://speech.ee.ntu.edu.tw/~tlkagk/courses.html>
  - <https://www.csie.ntu.edu.tw/~htlin/course/>
  - <https://www.csie.ntu.edu.tw/~yvchen/teaching>
  - <https://courses.cs.washington.edu/courses/cse599i/20au/> (generative models)
- Music information research
  - [https://www.audiolabs-erlangen.de/fau/professor/mueller/teaching/2023w\\_mpa](https://www.audiolabs-erlangen.de/fau/professor/mueller/teaching/2023w_mpa)
  - <https://musicinformationretrieval.com/>
  - <https://mac.kaist.ac.kr/~juhan/gct634/index.html>
  - <http://www.jordipons.me/apps/teaching-materials/>
  - <https://www.upf.edu/web/smc/audio-signal-processing-for-music-applications>

# Resources

- Conference proceedings
  - **Int'l Soc. Music Information Retrieval Conf. (ISMIR)**
  - Int'l Conf. Acoustic, Speech, and Signal Processing (ICASSP)
  - ACM MM, ACM ICMR, ACM SIGIR, IEEE ICME
- Transactions
  - IEEE Trans. Audio, Speech and Language Processing (TASLP)
  - IEEE Trans. Multimedia (TMM)
  - IEEE Trans. Signal Processing (TSP)

# Additional Enrollment

<https://forms.gle/8CMCrwmbh9GU69ph6>

- **Sign up** at NTU Cool (選課意願登記), if you are an NTU student
- **AND, Fill the form** before 23:59, **September 11** (Friday)
  - DL background
  - Music background
  - Ideas for final project
- Will announce the result before next **Monday**
  - Not first-come-first-serve; will read your submission
  - Will only send a mail to those selected

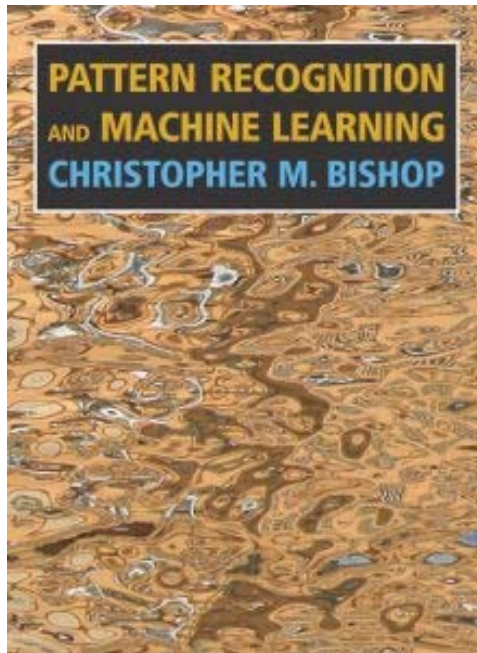


# Outline

- Intros of Music & AI
- Course logistics
- Fundamentals of musical audio
- **Appendix: ML 101**

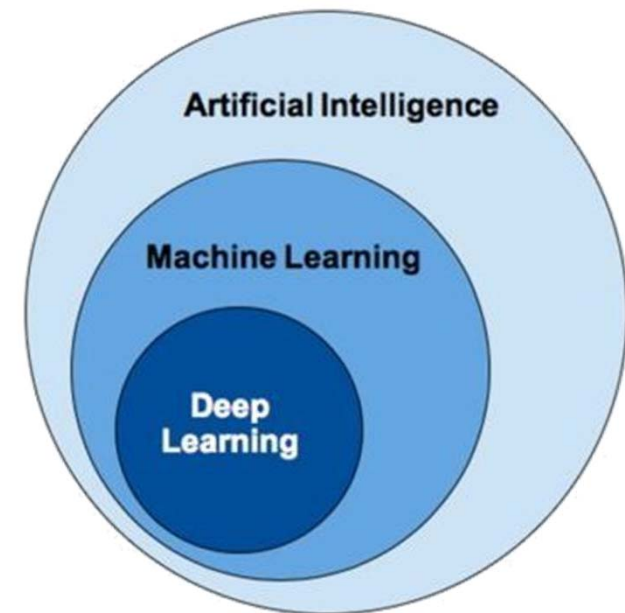
# Machine Learning

<https://www.microsoft.com/en-us/research/people/cmbishop/prml-book/>

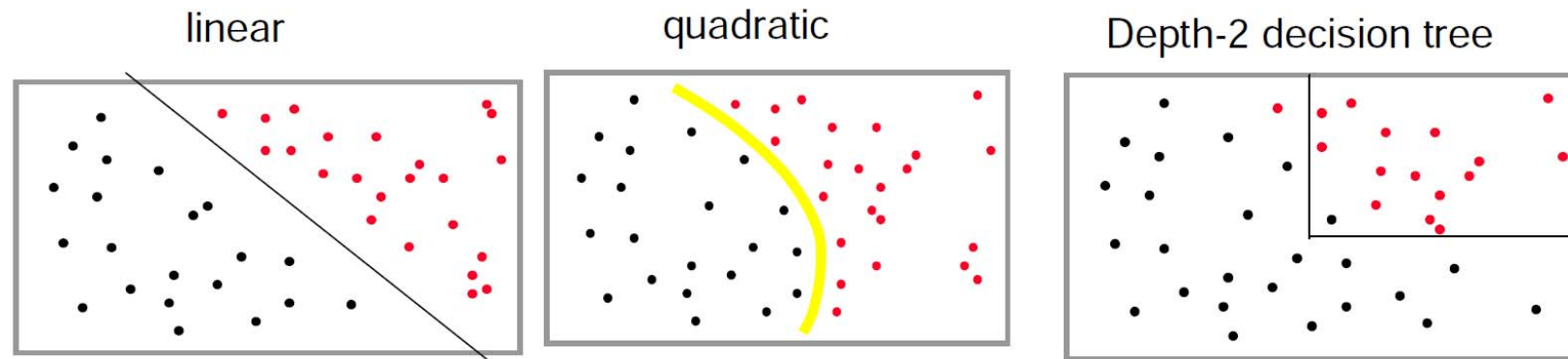


*Freely available*

1. Introduction
2. Probability Distributions
3. Linear Models for Regression
4. Linear Models for Classification
5. Neural Networks
6. ...



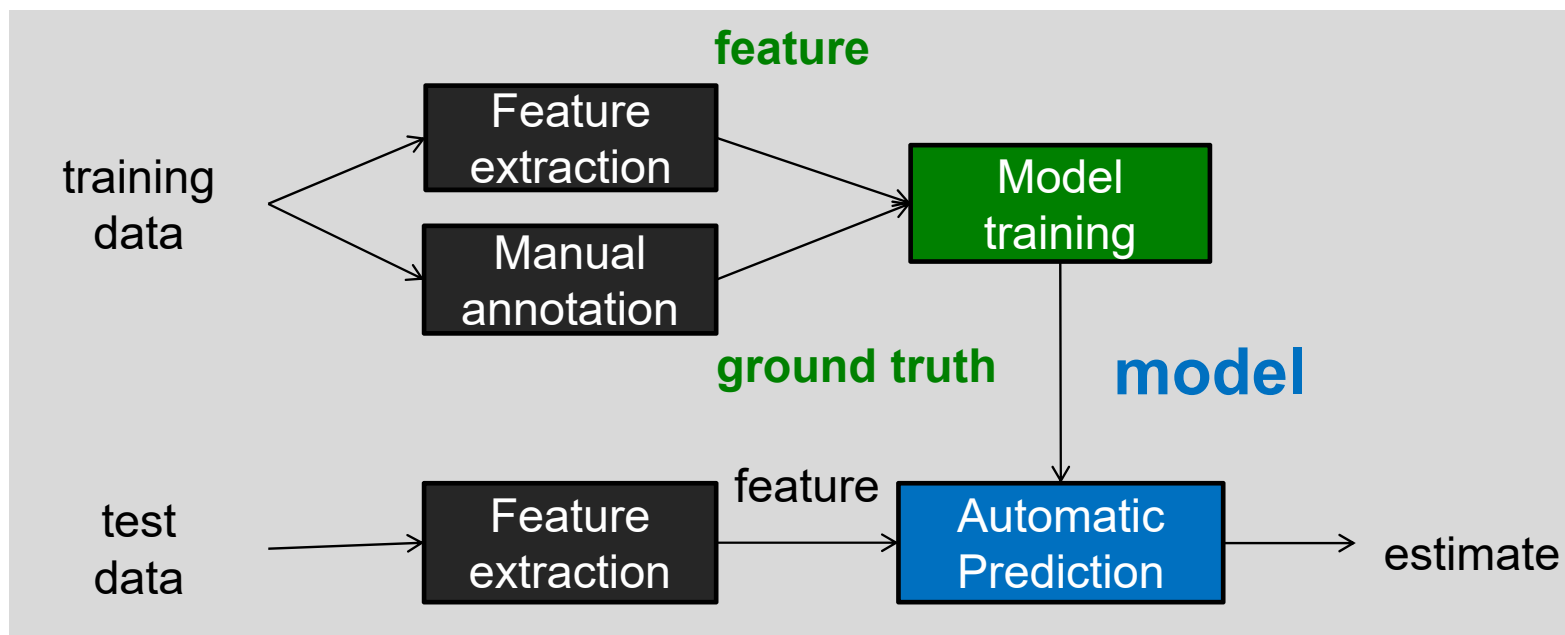
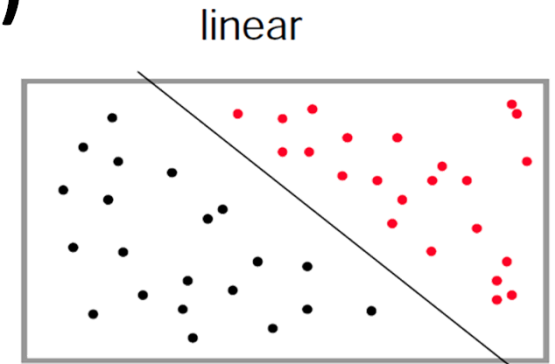
# Classification: Overview (1/2)



- Given:  $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ 
  - $\mathbf{x}_i \in \mathbb{R}^M$ : feature representation of a data instance
  - $y_i \in \{-1, +1\}$ : class label of that instance
- Goal
  - learn a separating hyperplane (decision boundary)

## Classification: Overview (2/2)

- Given:  $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ 
  - $\mathbf{x}_i \in \mathbb{R}^M$ : **feature** representation
  - $y_i \in \{-1, +1\}$ : class **label**



## Model Training (1/3)

1. Collect data ( $y$ ) and extract features ( $\mathbf{x}$ )
2. Build model: choose hypothesis class  $\mathcal{H}$  and loss function  $l$

– hypothesis class

- **linear:**  $\hat{y} = f(\mathbf{x}, \mathbf{w}, b) = \text{sgn}(\mathbf{w}^T \mathbf{x} + b)$
- **quadratic:**  $\hat{y} = f(\mathbf{x}, \mathbf{A}, \mathbf{w}, b) = \text{sgn}(\mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{w}^T \mathbf{x} + b)$

$$\text{sgn}(x) = \begin{cases} +1, & x \geq 0; \\ -1, & x < 0. \end{cases}$$

□ “parameters” to be *learned*:  $\theta = \{\mathbf{A}, \mathbf{w}, b\}$

## Model Training (2/3)

1. Collect data ( $y$ ) and extract features ( $x$ )
2. Build model: choose hypothesis class  $\mathcal{H}$  and loss function  $l$

### – loss function

- **indicator function:**  $l(y, \hat{y}) = 0$  if  $y = \hat{y}$ ;  
 $l(y, \hat{y}) = 1$  otherwise
  - **square loss:**  $l(y, \hat{y}) = (1 - y\hat{y})^2$
  - **hinge loss:**  $l(y, \hat{y}) = \max(0, 1 - y\hat{y})$
- ▣ while  $y \in \{-1, +1\}$ ,  $\hat{y}$  may not
  - ▣ hinge loss is used in support vector machine (SVM)

## Model Training (3/3)

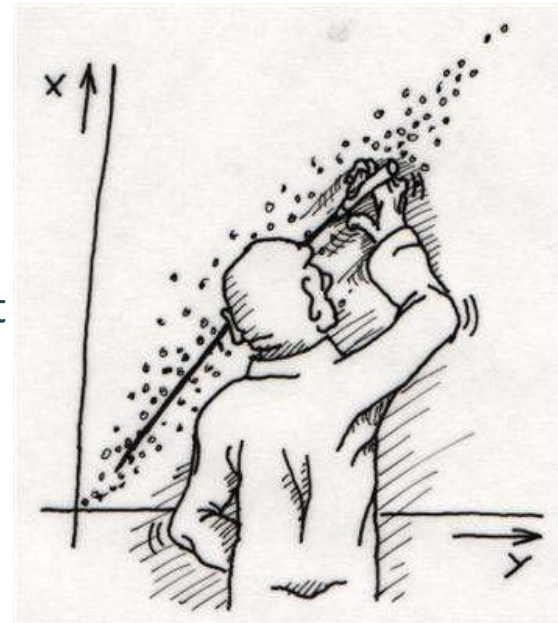
1. Collect data ( $y$ ) and extract features ( $x$ )
2. Build model: choose hypothesis class  $\mathcal{H}$  and loss function  $l$
3. **Optimization**: minimize the empirical loss
  - empirical loss:  $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i, \theta))$
  - optimization:  $\min_{\theta} \mathcal{L}(\theta) \Rightarrow$  by computing  $\frac{\partial \mathcal{L}(\theta)}{\partial \theta}$

## More on Loss Functions (1/4)

- Different loss function leads to different machine learning problems
  - **classification**:  $l(y, \hat{y}) = (1 - y\hat{y})^2$
  - **regression**:  $l(y, \hat{y}) = (y - \hat{y})^2$

Given  $N$  inputs  $(\mathbf{x}_i, y_i)$ ,  $1 \leq i \leq N$ , where  $\mathbf{x}_i$  is feature and  $y_i$  is the numerical value to be predicted, train a regression model  $f(\cdot)$  such that the **mean squared error (MSE)** is minimized

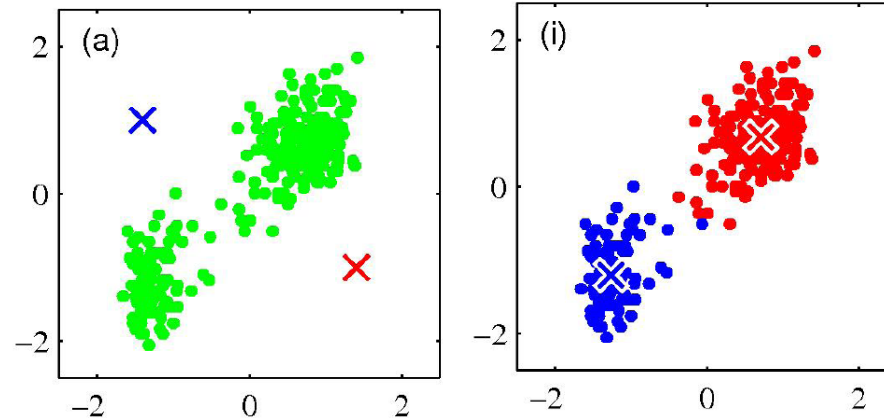
$$\min_f \frac{1}{N} \sum_{i=1}^N (y_i - f(\mathbf{x}_i))^2$$



## More on Loss Functions (2/4)

- Different loss function leads to different machine learning problems
  - **classification**:  $l(y, \hat{y}) = (1 - y\hat{y})^2$
  - **regression**:  $l(y, \hat{y}) = (y - \hat{y})^2$
  - **clustering** (an unsupervised task)

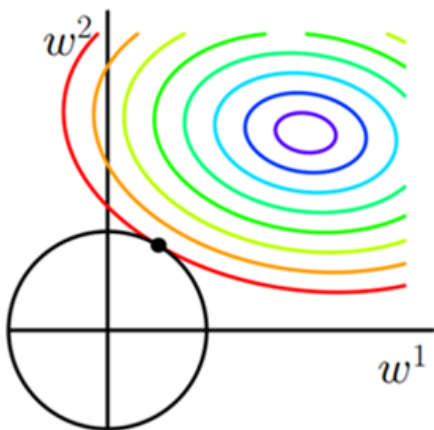
$$\mathcal{L}(\Delta) = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$$



## More on Loss Functions (3/4)

- Adding regularizers

- original:  $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2$
- modified:  $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2$



(b)  $\ell_2$ -ball meets quadratic function.  $\ell_2$ -ball has no corner. It is very unlikely that the meet-point is on any of axes.

$$\|\mathbf{w}\|_2^2 = \mathbf{w}^T \mathbf{w} = \sum_{j=1}^M w_j^2$$

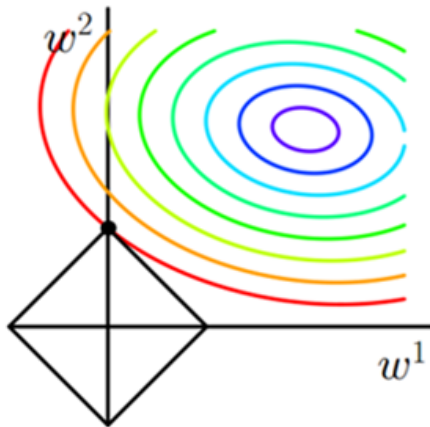
prefer  $\mathbf{w} = [1.5, -0.5, 0.8, -1.0]^T$   
over  $\mathbf{w} = [9.5, -8.0, 2.5, -3.0]^T$ ,  
despite that the latter has smaller MSE

- $\lambda \geq 0$  is a parameter to be tuned (i.e. empirically determined)

## More on Loss Functions (4/4)

- Adding regularizers

- modified:  $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2$
- modified:  $\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1$



(a)  $\ell_1$ -ball meets quadratic function.  $\ell_1$ -ball has corners. It's very likely that the meet-point is at one of the corners.

$$\|\mathbf{w}\|_1 = \sum_j |w_j|$$

prefer  $\mathbf{w} = [1.5, 0, 0, -2.0]^T$   
over  $\mathbf{w} = [1.5, -0.5, 0.8, -1.0]^T$ ,  
despite that the latter has smaller MSE

- L1 is the only norm that is both convex and sparsity promoting

## Example: Ridge Regression (for Regression)

- Loss function

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\text{or, } \mathcal{L}(\mathbf{w}) = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}$$

$\begin{matrix} \swarrow & \swarrow & \searrow \\ N \times 1 & N \times M & M \times 1 \end{matrix}$

- Optimization

$$\frac{\partial \mathcal{L}(\mathbf{w})}{\partial \mathbf{w}} = \underset{M \times N}{2\mathbf{X}^T} \underset{N \times 1}{(\mathbf{X}\mathbf{w} - \mathbf{y})} + 2\lambda \underset{M \times 1}{\mathbf{w}} \stackrel{\text{let}}{=} \mathbf{0}$$

$$\therefore \mathbf{X}^T \mathbf{y} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{w}$$

$$\therefore \mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

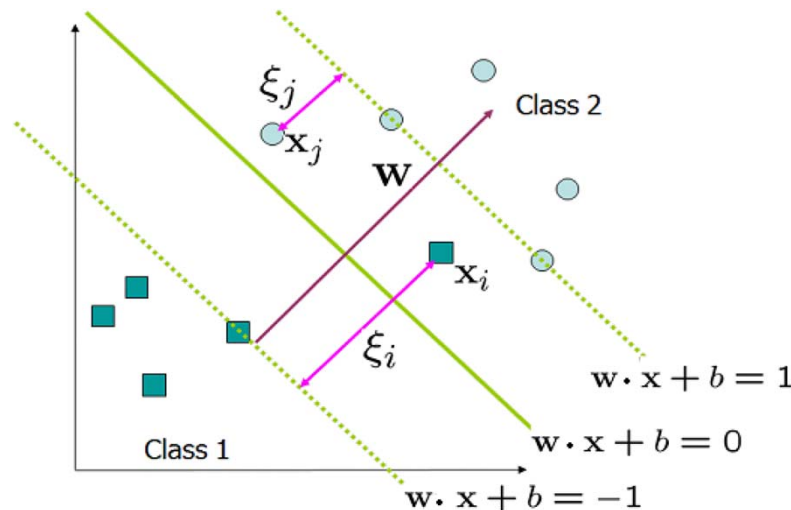
## Example: Support Vector Machine (1/3)

- Loss function

$$\min_{\mathbf{w}, \xi_i} \|\mathbf{w}\|_2 + C \sum_{i=1}^N \xi_i$$

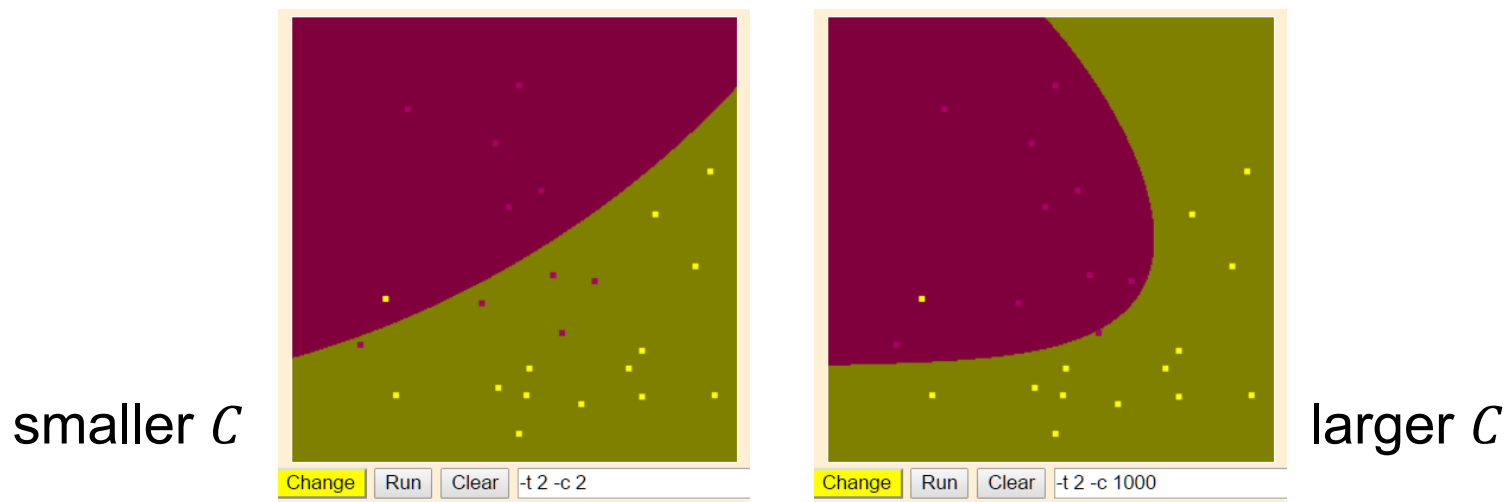
subject to  $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$  for  $i = 1 \dots N$

- the idea of hinge loss  $\max(0, 1 - y\hat{y})$  is also to have  $y\hat{y} \geq 1$
- the slack variables  $\xi_i \geq 0$  allow for errors



## Example: Support Vector Machine (2/3)

- Loss function:  $\min \|\mathbf{w}\|_2 + C \sum_{i=1}^N \xi_i$ 
  - smaller  $C$ : reduced model complexity
  - larger  $C$  : less misclassification
- Demo: <https://www.csie.ntu.edu.tw/~cjlin/libsvm/>



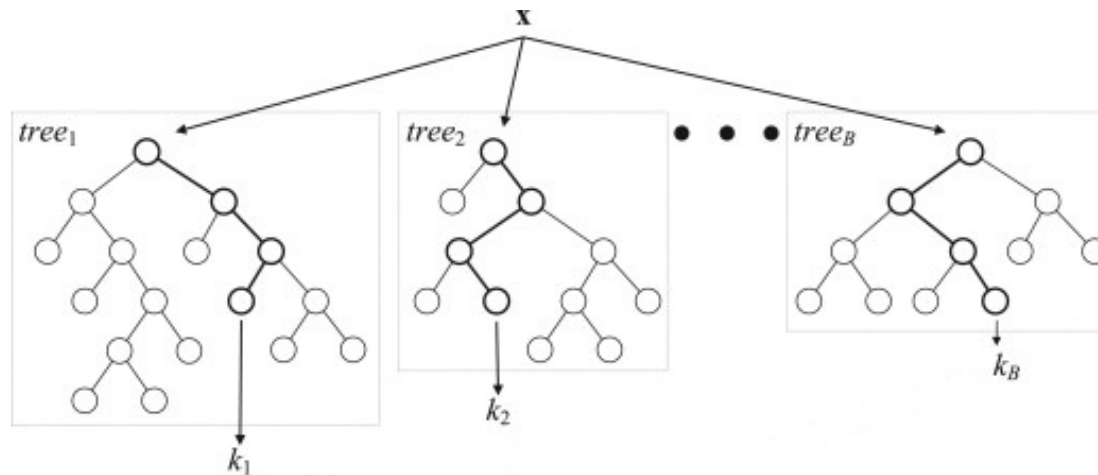
## Example: Support Vector Machine (3/3)

- Different kernel functions of SVM
  - the **RBF** kernel (-t 2) is often used for nonlinear SVM; important parameters:  $C$  and  $\gamma$

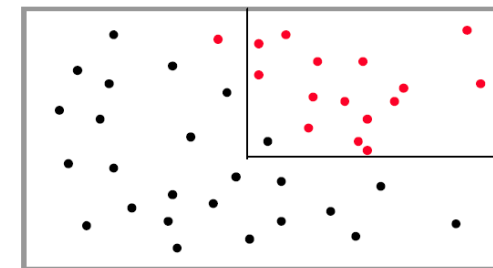
```
options:
-s svm_type : set type of SVM (default 0)
    0 -- C-SVC
    1 -- nu-SVC
    2 -- one-class SVM
    3 -- epsilon-SVR
    4 -- nu-SVR
-t kernel_type : set type of kernel function (default 2)
    0 -- linear:  $u \cdot v$ 
    1 -- polynomial:  $(\gamma u \cdot v + \text{coef0})^{\text{degree}}$ 
    2 -- radial basis function:  $\exp(-\gamma |u-v|^2)$ 
    3 -- sigmoid:  $\tanh(\gamma u \cdot v + \text{coef0})$ 
-d degree : set degree in kernel function (default 3)
-g gamma : set gamma in kernel function (default  $1/\text{num\_features}$ )
-r coef0 : set coef0 in kernel function (default 0)
-c cost : set the parameter C of C-SVC, epsilon-SVR, and nu-SVR (default 1)
```

# Example: Random Forest

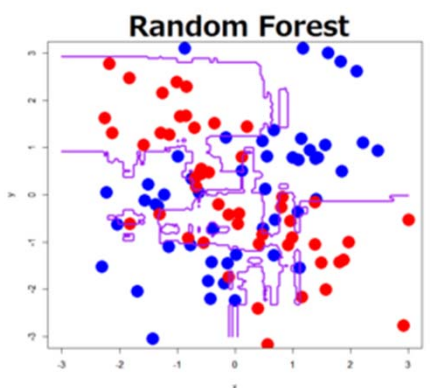
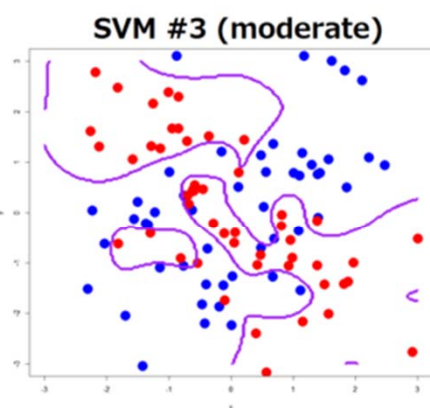
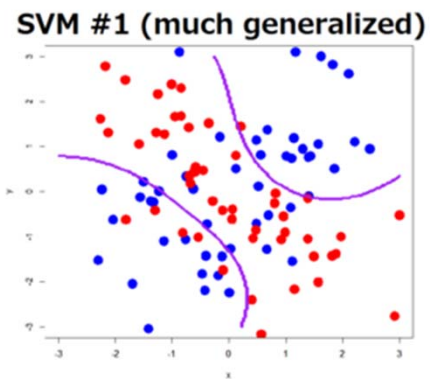
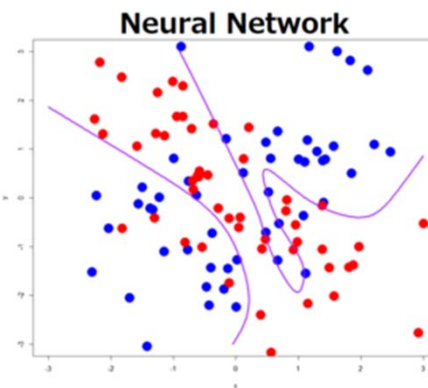
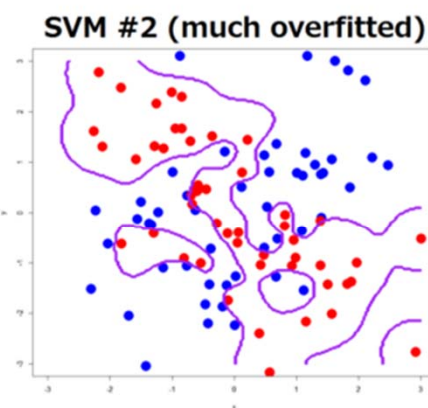
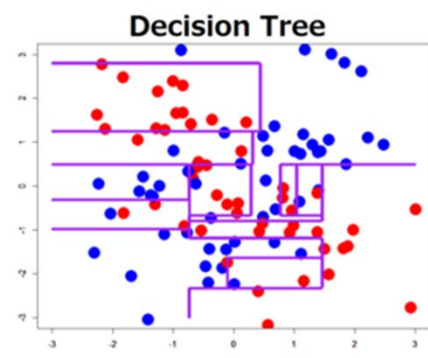
- Ensemble of decision trees
  - important parameters: number of trees, tree depth
  - strength: interpretability, non-linear



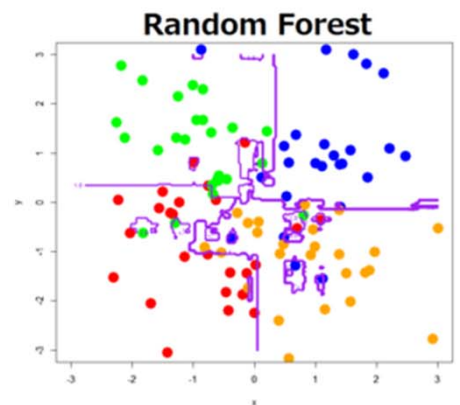
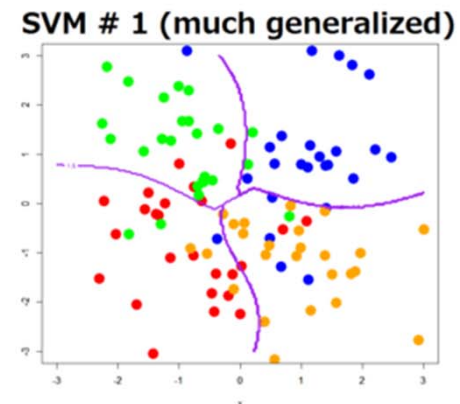
Depth-2 decision tree



*binary classification*

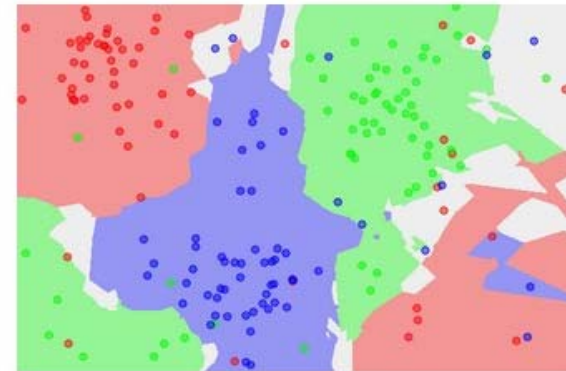
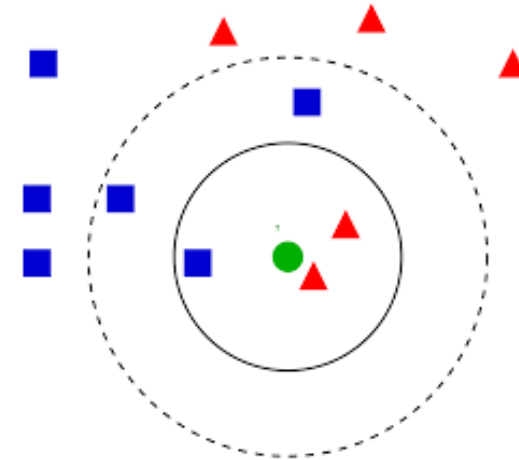


*multi-class classification*



## Example: k-Nearest Neighbors

- Classify an instance by a **majority vote** of the class membership of its  $k$  nearest neighbors
  - $k$  is a positive integer
  - performance depends on the value of  $k$  and the features
  - lazy learning
  - nonlinear classifier



# Evaluation (1/3)

- Model training
  1. Collect data and extract features
  2. Build model: choose hypothesis class and loss function
  3. Optimization: minimize the empirical loss
- **Evaluation**
  4. Evaluate on unseen, testing data

## Evaluation (2/3)

- Performance measures
  - classification accuracy (CA)
  - per-class precision (P), recall (R), F-score (F)
  - confusion table

|              |        | Predicted |     |        |
|--------------|--------|-----------|-----|--------|
|              |        | Cat       | Dog | Rabbit |
| Actual class | Cat    | 5         | 3   | 0      |
|              | Dog    | 2         | 3   | 1      |
|              | Rabbit | 0         | 2   | 11     |

$$CA = (5+3+11)/(5+3+0+2+3+1+0+2+11)$$

$$P(cat) = 5/(5+2)$$

$$R(cat) = 5/(5+3)$$

# Evaluation (3/3)

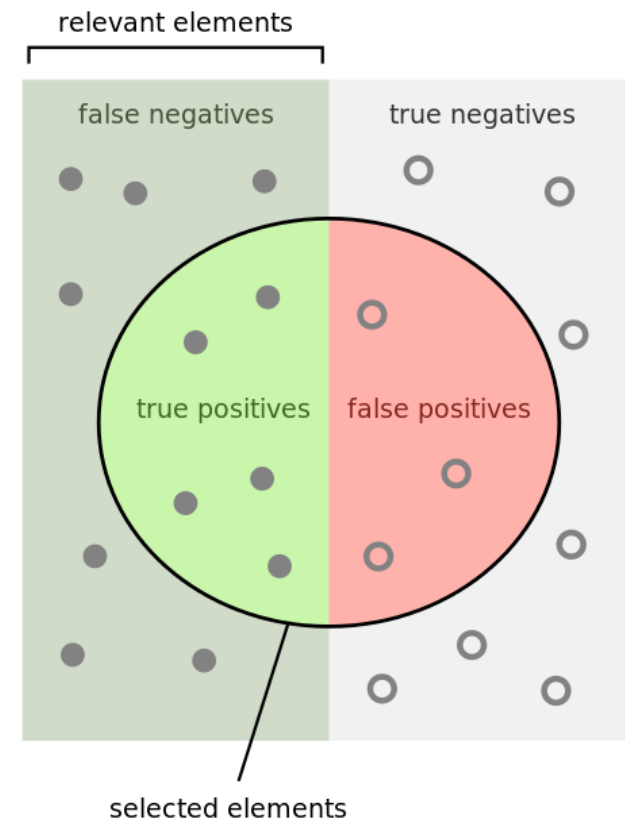
- Performance measures
  - precision (**P**):  $TP/(TP+FP)$
  - recall (**R**):  $TP/(TP+FN)$
  - F-score (**F**):  $2PR/(P+R)$

How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$


How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

# Train, Validation, Test (1/2)

- Test set is unseen, how to evaluate our classifier?
  - use part of the training set as the “**validation set**”
  - e.g. 80% for training, 20% for validation
  - use the validation set to: 1) tune the parameters, 2) select classification algorithms, and 3) select features
- **K-fold Cross validation**
  - divide the data into  $K$  folds, use one fold for validation and the rest of training, repeat  $K$  times and report the average result

# Train, Validation, Test (2/2)

- **Model overfitting**
  - training accuracy  $\neq$  validation accuracy
  - that's why we need a balance between model complexity and training error
- **Cross-dataset generalizability**
  - validation accuracy  $\neq$  test accuracy
- **Cheating issue**
  - make sure there is **no overlaps** between the training and test sets
  - **song-level split** (preferred) vs. chunk-level split

# Data Labeling (1/2)

- Collect data and extract features
- How to get the labels?
  - **existing datasets**
    - due to copyright restrictions, it's not easy to get the audio files
  - **crawl the web**
    - unsure of the quality; do not know how the labeling was done
  - **manually annotate**
    - labor intensive and time consuming but can be worthwhile

# Data Labeling (2/2)

- Issues to be taken care of while building a new dataset
  - 1. be aware of “confounds”**
    - Example 1: Jazz from the 60’s vs. contemporary Rock
    - Example 2: Happy songs from artist 1 vs. sad songs from artist 2
  - 2. try to make the annotation process easier**
    - binary decision -> multiple choices -> rating
    - gamification
  - 3. clear instructions**
  - 4. check for reliability/consistency**

# Feature Extraction and Processing

- Collect data and extract features
- The usual pipeline
  - frame-level **feature extraction**
  - **pooling** (from frame-level to clip-level)
  - **feature normalization**
- Classification tasks in MIR
  - **clip-level prediction**: genre classification
  - **frame-level prediction**: onset detection, pitch detection
  - for clip-level prediction, pooling is often used

# Feature Pooling

- Rationale
  - thousands of frames per clip → thousands of feature vectors per clip
  - we can either train a frame-level classifier, but eventually still have to integrate the result to the clip-level
  - or, we can integrate the feature vectors before training the classifier
- Approach
  - take statistics such as **mean** and **standard deviation**

# Feature Normalization/Scaling

- For each individual feature dimension
  - **linear normalization:**  $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$
  - **z-score normalization:**  $x' = \frac{x - \text{mean}(x)}{\text{std}(x)}$
  - the values  $\text{mean}(x)$  and  $\text{std}(x)$  are computed from all the *training* instances (i.e. *not* including the validation and the test sets)
- For each individual instance
  - **scaling to unit length:**  $x' = \frac{x}{|x|}$

# Early Fusion or Late Fusion

- You can extract features of different types to characterize different aspects of the signal
  - timbre, pitch, rhythm, lyrics, etc
- **Early fusion** (feature-level fusion)
  - concatenate them into a single feature vector then train a classifier
- **Late fusion** (decision-level fusion)
  - train a classifier for each feature type, and then aggregate the estimate of these classifiers (e.g., majority vote or average) to make the final decision

# Feature Selection

- Not all features would be relevant
- Feature selection according to
  - domain knowledge
  - empirical observation at the feature matrix (e.g., remove a feature if there is NaN or outlier [or replace the NaNs by the average])
  - statistical methods

# Library: Torchaudio

[https://pytorch.org/audio/0.11.0/tutorials/audio\\_feature\\_extractions\\_tutorial.html](https://pytorch.org/audio/0.11.0/tutorials/audio_feature_extractions_tutorial.html)

```
waveform, sample_rate = get_speech_sample()

n_fft = 1024
win_length = None
hop_length = 512

# define transformation
spectrogram = T.Spectrogram(
    n_fft=n_fft,
    win_length=win_length,
    hop_length=hop_length,
    center=True,
    pad_mode="reflect",
    power=2.0,
)

# Perform transformation
spec = spectrogram(waveform)

print_stats(spec)
plot_spectrogram(spec[0], title="torchaudio")
```

# Python: Librosa

<https://librosa.org/doc/main/feature.html>

## [-] Feature extraction

- [+] Spectral features
- [+] Rhythm features
- [+] Feature manipulation
- [+] Feature inversion

Onset detection

Beat and tempo

Spectrogram decomposition

Effects

Temporal segmentation

Sequential modeling

Utilities

# Python: scikit-learn

<https://scikit-learn.org/1.4/tutorial/index.html>

 [Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More ▾](#)

[Prev](#) [Up](#) [Next](#)

**scikit-learn 1.4.2**  
[Other versions](#)

Please [cite us](#) if you use the software.

**Welcome to scikit-learn**  
**scikit-learn Tutorials**  
An introduction to machine learning with scikit-learn  
A tutorial on statistical-learning for scientific data processing  
Working With Text Data  
Choosing the right estimator  
External Resources, Videos and Talks  
**Getting Started**  
**User Guide**  
**Glossary of Common Terms and API Elements**  
**Examples**

This is documentation for an old release of Scikit-learn (version 1.4). Try the [latest stable release](#) (version 1.5) or [development](#) (unstable) versions.

**scikit-learn Tutorials**

**An introduction to machine learning with scikit-learn**

- Machine learning: the problem setting
- Loading an example dataset
- Learning and predicting
- Conventions

# Practical Issues 1: Multi-label Classification

- A data instance can be labeled with more than one classes
- *Multi-class* classification (one-hot; one out of many) vs. *multi-label* classification (multi-hot)
- Harder for ML (may need multiple binary classifiers) but simple for DL: it's just a matter of changing the loss function
  - Multi-class classification: **categorical cross entropy** (CE)
  - Multi-label classification: **binary cross entropy** (BCE)

## Practical Issues 2: Data Imbalance

- Simply predicting everything as the majority class would already give you good accuracy
- A naïve workaround: **subsampling the majority class**

## Practical ML Hints (not for DL)

- Look into the features
- Remember to normalize the features
- Use validation set to properly tune the parameters
- Nonlinear classifier usually performs better, but also requires longer training time
- Use a subset of data to fast prototype your ideas first, before using the full set for more complete experiments
- Study the classification result (e.g., confusion table)